AI Changing Software Engineering



agenda

- AI Hype
- AI as a continuation of software engineering evolution
  - What is software engineering?
  - Software evolution
- AI impact on software engineering
  - Software development
    - Productivity and efficiency
    - AI code assistant best practices
    - AI risks
  - Future



AI generated code

Satya Nadella says as much as 30% of Microsoft code is written by AI

PUBLISHED TUE, APR 29 2025•9:33 PM EDT | UPDATED TUE, APR 29 2025•9:58 PM EDT



Jordan Novet @IN/JORDANNOVET/ | Jonathan Vanian @IN/JONATHAN-VANIAN-B704432/

SHARE

TRENDING NOW

50% of Our Company's Code Is Now AI-Generated



Walter Guevara Follow 4 min read · May 22, 2025

Forbes

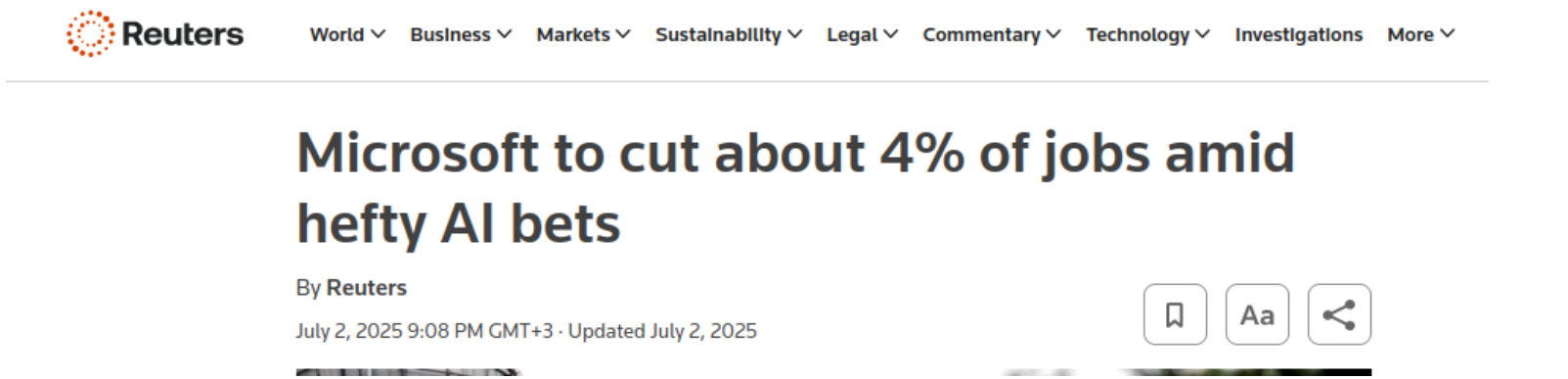
LEADERSHIP › CAREERS

AI Writes Over 25% Of Code At Google—What Does The Future Look Like For Software Engineers?

By Jack Kelly, Senior Contributor. ⓘ Jack Kelly covers career growth, job mark... Follow Author

Published Nov 01, 2024, 06:30am EDT

layoff


Reuters
World
Business
Markets
Sustainability
Legal
Commentary
Technology
Investigations
More
Microsoft to cut about 4% of jobs amid hefty AI bets
By Reuters
July 2, 2025 9:08 PM GMT+3 · Updated July 2, 2025
Aa

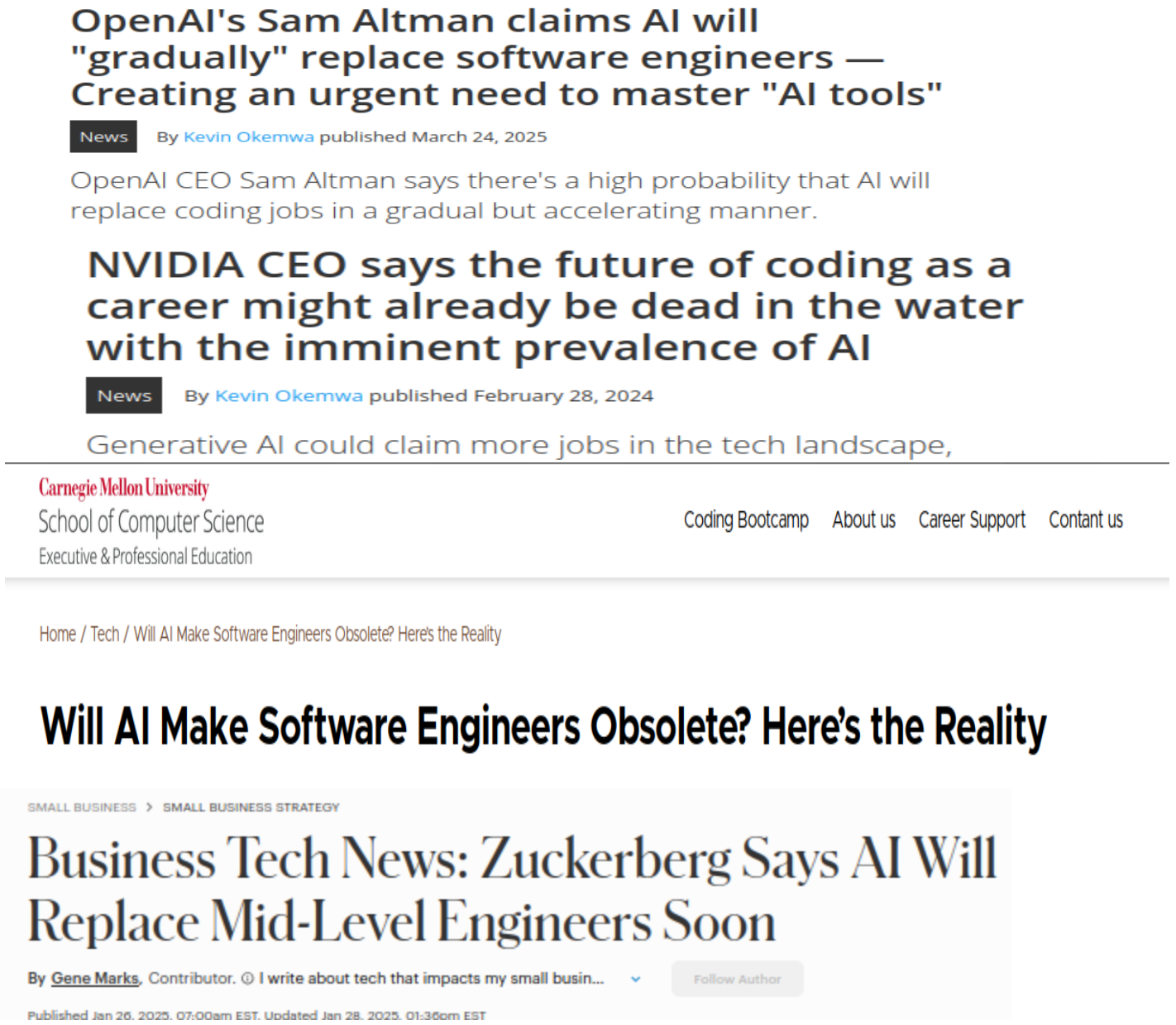

AI-driven job cuts may be underreported: Challenger
Only 75 job cuts by U.S.-based employers in the first half of the year were explicitly


Hundreds face layoffs as Amdocs centralizes AI in new division
GenAI & Data unit aims to make artificial intelligence part of


capacity
a techoraco brand
News
Industry Sectors


Tech layoffs accelerate as industry retools for AI and efficiency
Saf Malik
September 17, 2025 10:38 AM

Replacement



OpenAI's Sam Altman claims AI will "gradually" replace software engineers — Creating an urgent need to master "AI tools"

News By Kevin Okemwa published March 24, 2025

OpenAI CEO Sam Altman says there's a high probability that AI will replace coding jobs in a gradual but accelerating manner.

NVIDIA CEO says the future of coding as a career might already be dead in the water with the imminent prevalence of AI

News By Kevin Okemwa published February 28, 2024

Generative AI could claim more jobs in the tech landscape,

Carnegie Mellon University
School of Computer Science
Executive & Professional Education

Coding Bootcamp About us Career Support Contant us

Home / Tech / Will AI Make Software Engineers Obsolete? Here's the Reality

Will AI Make Software Engineers Obsolete? Here's the Reality

SMALL BUSINESS > SMALL BUSINESS STRATEGY

Business Tech News: Zuckerberg Says AI Will Replace Mid-Level Engineers Soon

By Gene Marks, Contributor. I write about tech that impacts my small busin... Follow Author

Published Jan 26, 2025, 07:00am EST, Updated Jan 28, 2025, 01:36pm EST

Memes


SELL ME THIS PEN
ITS AI POWERED


AI IS
THE FUTURE
AI IS GOING
TO KILL US ALL
TECH BROS
Did you just take both pills?

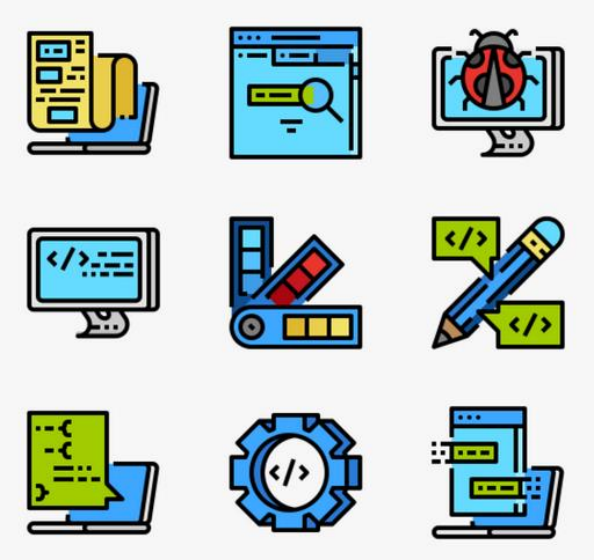

WHAT ARE YOU STUDYING?
PROGRAMMING

Software engineering

> “Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.”
>
> **IEEE Standard Glossary**

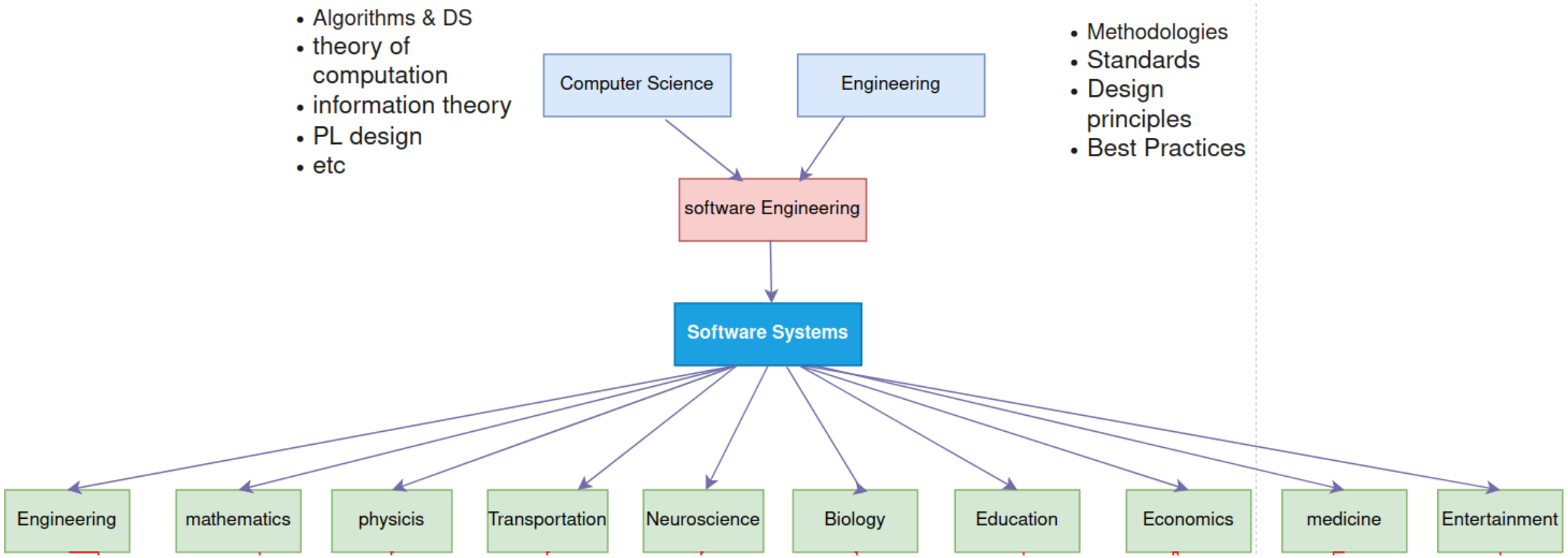
"software engineering is a branch of both computer science and engineering focused on designing, developing, testing, and maintaining software applications. It **develops software systems** by **applying engineering principles and computer programming expertise** to **meet user needs**"

Wikipedia

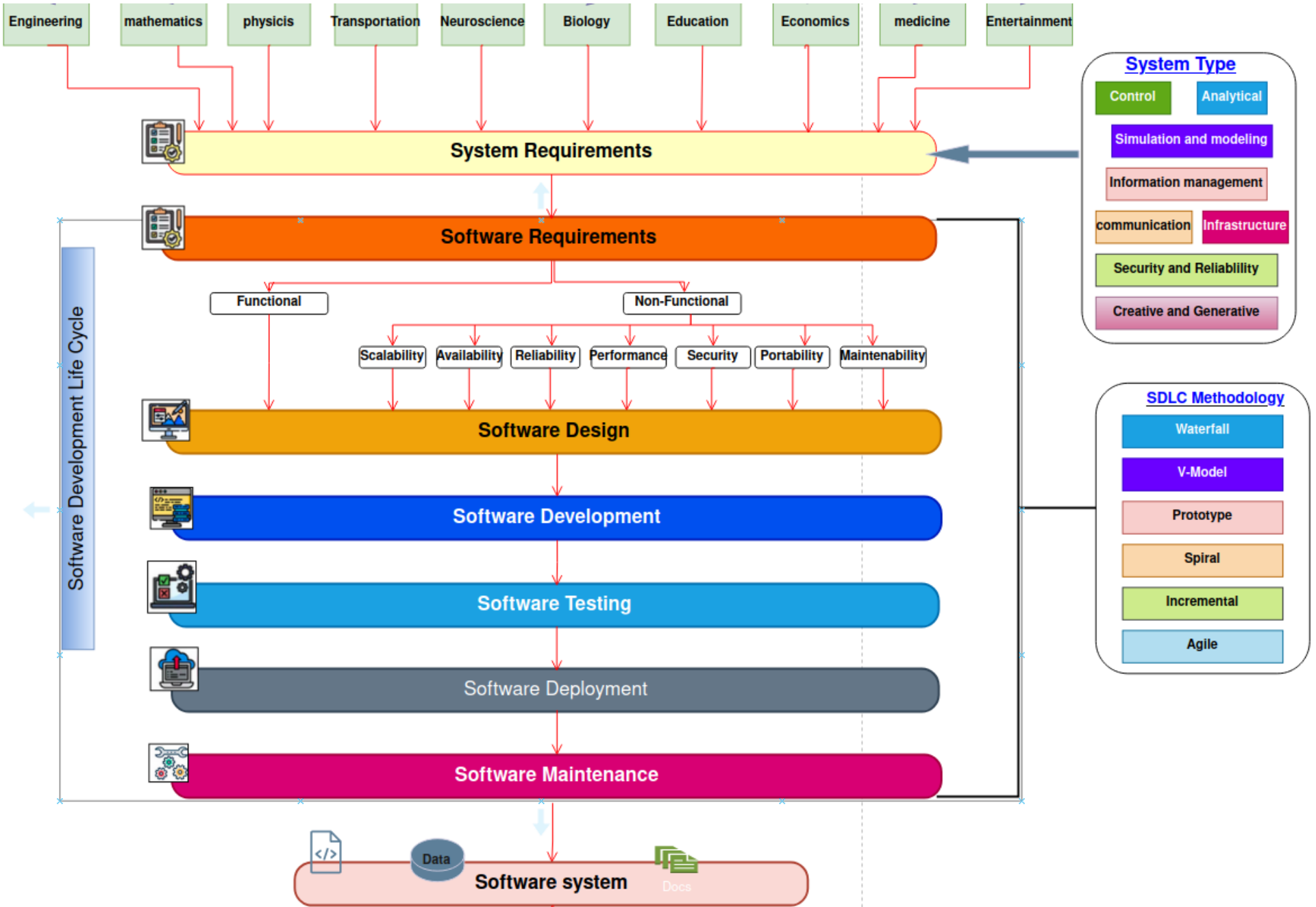


"Our civilization is as reliant on software as it is on water"
Bjarne Stroustrup

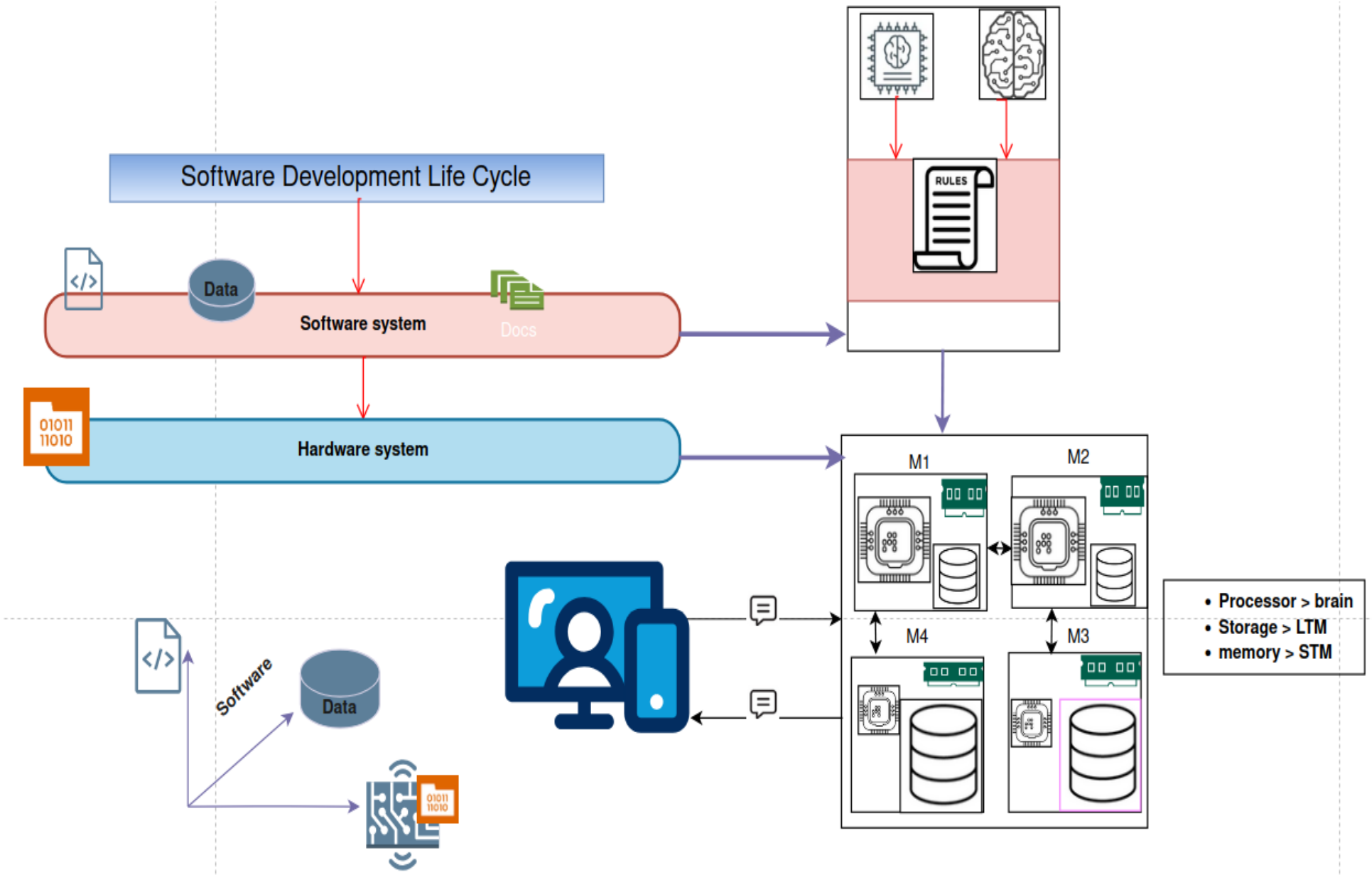
Software engineering



SDLC


Engineering
mathematics
physicis
Transportation
Neuroscience
Biology
Education
Economics
medicine
Entertainment
System Requirements
Software Requirements
Functional
Non-Functional
Scalability
Availability
Reliability
Performance
Security
Portability
Maintenability
Software Design
Software Development
Software Testing
Software Deployment
Software Maintenance
Software Development Life Cycle
Data
Software system
Docs
System Type
Control
Analytical
Simulation and modeling
Information management
communication
Infrastructure
Security and Reliablility
Creative and Generative
SDLC Methodology
Waterfall
V-Model
Prototype
Spiral
Incremental
Agile

Computer system



Brain: Processor evolution

1944



IBM Harvard Mark I
3,500 relays
Addition: ~4/sec

1945



MSEE: ENIAC
17,468 vacuum tubes
Addition: ~5K/sec

1959



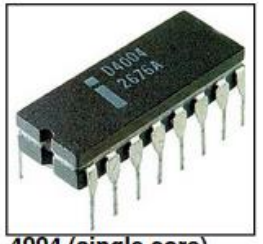
IBM 1401
~10K transistors (BJT)
Addition: ~11K/sec

1964



IBM System/360 (ICs)
~600K transistors (MOS)
Addition: ~100K/sec

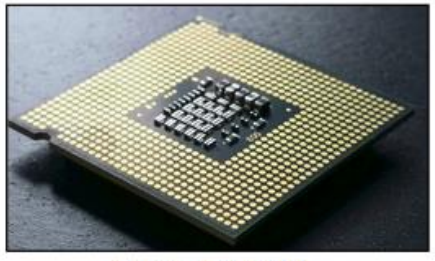
1971



4004 (single core)
~2.3K transistors (MOS)
Addition: ~93K/sec
clock: ~740KHZ

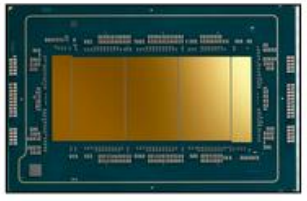


Field-Programmable-Gate-Array
(FPGA)



System-On-Chip
(SOC)

2024



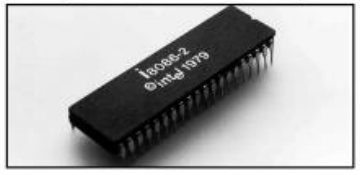
Granite Rapids (128 cores)
~100B transistors (MOS)
Addition: ~10T/sec
clock: ~4GHZ

2005

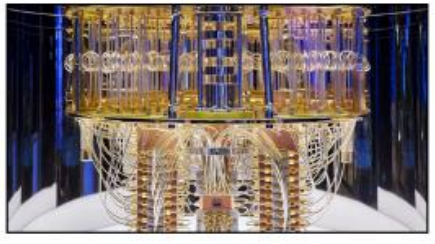


Pentium D (2 cores)
~230M transistors (MOS)
Addition: ~10B/sec
clock: ~3GHZ

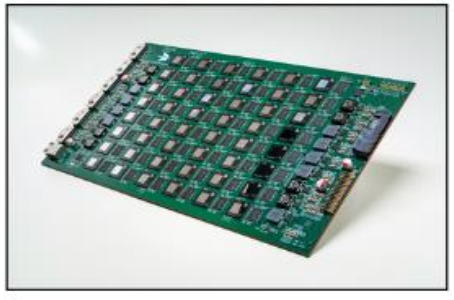
1978



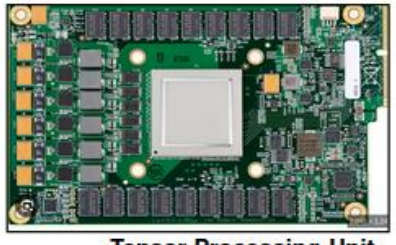
8086 (single core)
~29K transistors (MOS)
Addition: ~1M/sec
clock: ~10MHZ



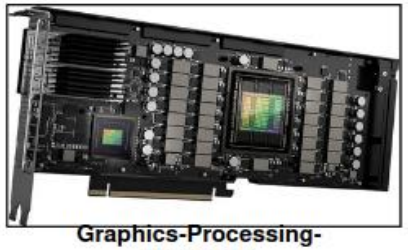
Quantum Computer



Neuromorphic Chip



Tensor-Processing-Unit
(TPU)



Graphics-Processing-Unit(GPU)

Short-term memory: Memory evolution

Evolution of Memory Devices

1946



Williams Tube
~2K-8K

1954



IBM 704 : magnetic core
~18KB

1970



Intel 1103 DRAM: Transistor
~1kB

2002



Hynix PC2100R DDR1
1GB

2024



Samsung DDR5
1TB

2019



Samsung M386ABG4 DDR4-3D
256GB

2018



Samsung M393T51 DDR4
64GB

2006



Samsung M393T51 DDR2
4GB

Long-term memory: Storage evolution

Evolution of Storage Devices



1956	1966	1989	1999	2013	2025
IBM 305 RAMAC 350	**DEC RS0B**	**Western Digital WD93044-A**	**IBM Microdrive**	**Samsung 840 EVO**	**Samsung PM1743 PCIe 5.0 SSD**
Magnetic Disk Drive	Magnetic Drum	3.5" HDO (IDE)	1-Inch HDD	**1 TB**	NVMe SSD
5 MB	**256 KB**	**40 MB**	**340 MB**	SATA SSD	**30 TB**
From first HDD to 50 platters	Early rotating drum memory	IDE/ATA interfe ce introduced	**340 MB** Miniature HDD for cameras	**1 TB** Flash storagee reaches terabyte capacity	Enterprise-class NVMe Gen 5 drive



Hyperscale data centers (AWS, Google, Microsoft, Meta) – exabytes (10^{18} bytes)

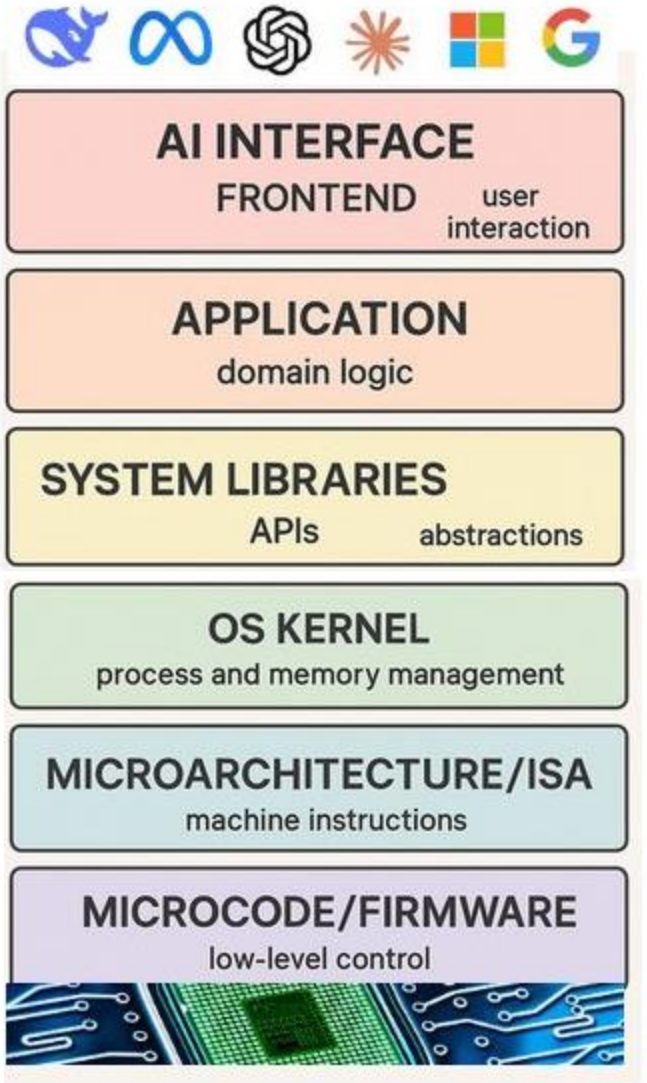
it can hold **5–20 EB** of storage across tens of thousands of drives and SSDs.

(1 EB = 1,000 PB = 1 million TB.)

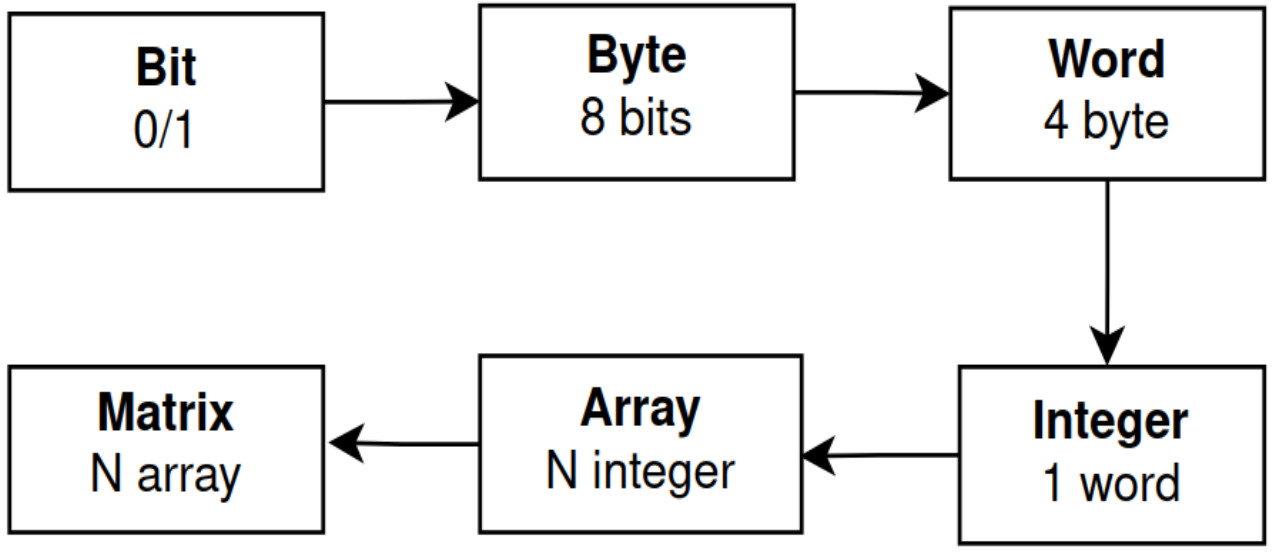
Digital Pale blue dot.

Software abstraction and evolution

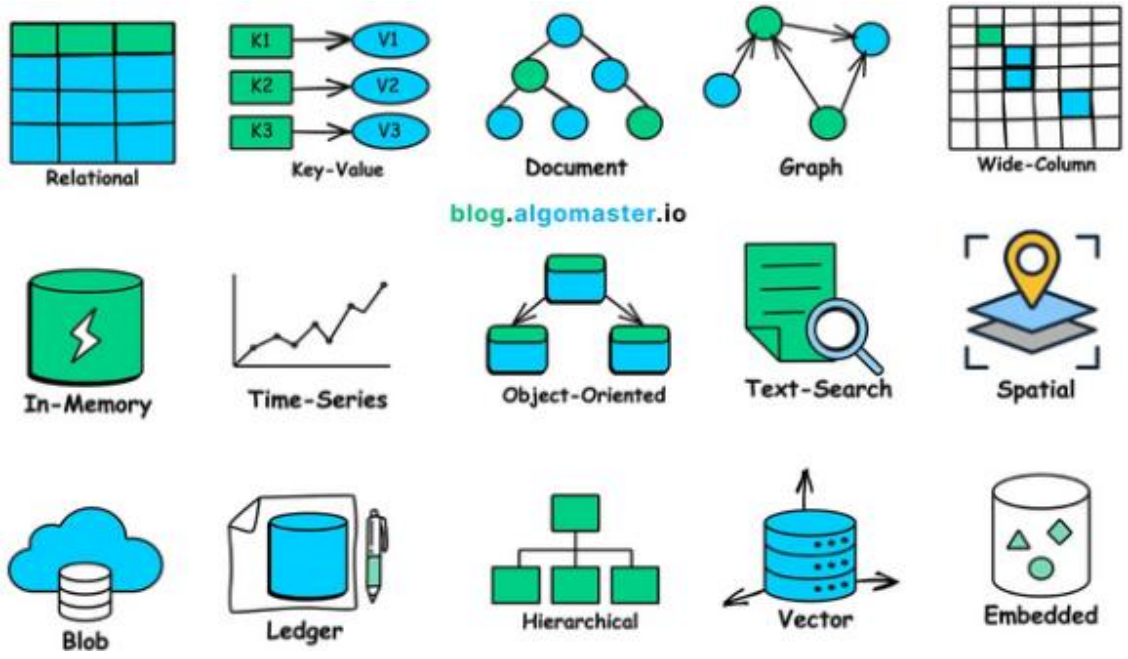
"Our civilization runs on software, and software runs on abstractions" **Bjarne Stroustrup**



Memory (STM) and storage (LTM) abstraction



Data abstraction



Data model

Programming abstraction

“Programming is the art of telling another human being what one wants the computer to do.”

Donald Knuth



Intention expression

Human World

Machine World



Analyze feature X and make a draft implemention





fetch data from API and save to CSV





```
print("Hello World")
```





```
#include <stdio.h>

int main(void)
{
    printf("Hello World\n");
    return 0;
}
```





```
.model small
.stack 100h
.data
msg db "Hello World",'$'

.code
main proc
        mov ax,@data
        mov ds, ax

        mov dx,offset msg
        mov ah,09
        int 21h

        mov ax,4c00h
        int 21h
main endp
end main
```





```
10111000 00000001
00000000 00000000
00000000 10111111
00000001 00000000
00000000 00000000
00000000 01001000
10001101 00110101
00010000 00000000
00000000 10111010
00001110 00000000
00000000 00000000
00000000 00001111
00000101 10111000
00111100 00000000
00000000 00000000
00110001 11111111
00001111 00000101
01001000 01100101
01101100 01101100
01101111 00101100
00100000 01110111
01101111 01110010
01101100 01100100
00100001 00001010
```



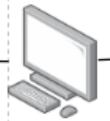












Software architecture abstraction

- Learn the core principles of technologies and their trade-offs
- Use the right tools for the right job in the right way





Software projects





















Who writes the code?



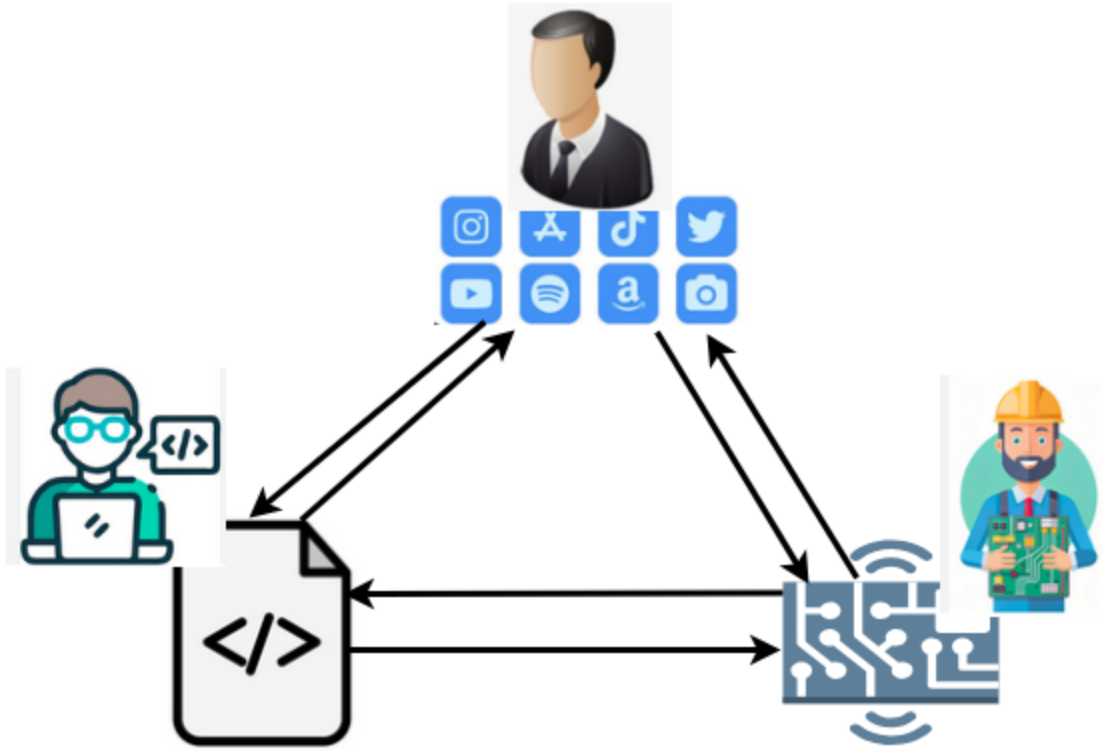






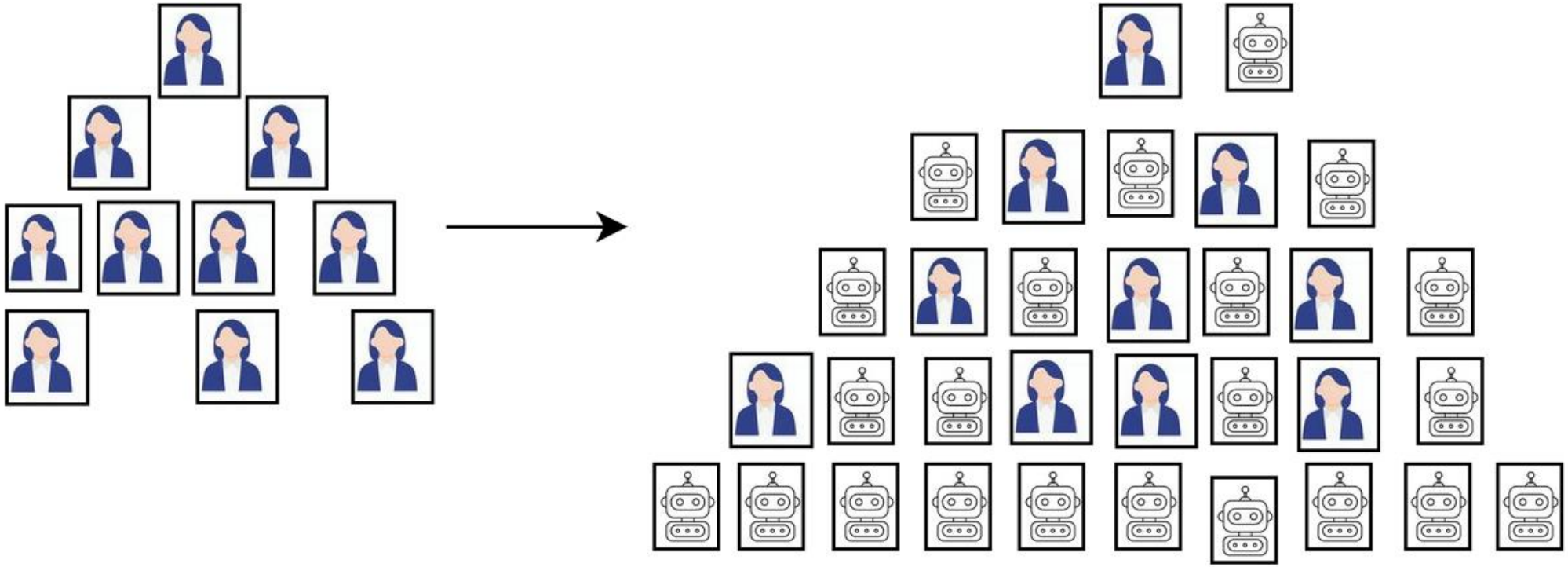
Mutual impact of people and technology

In the dance of innovation, software, hardware, and people evolve together. Each shaping the boundaries and possibilities of the others.



"We shape our tools and then our tools shape us" Marshal Mcluhan

The Human–AI Organization Chart



AI impact on SW engineering

- Intelligent information search

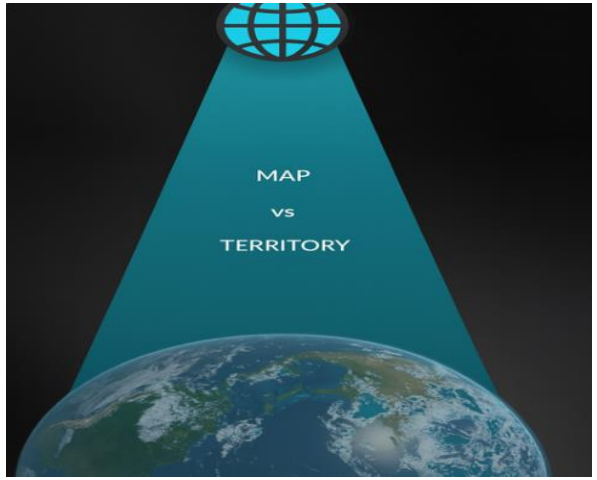




- Exploration and Ideation (Brainstorming)



- Building conceptual models



- Role playing



- Writing



- Learning

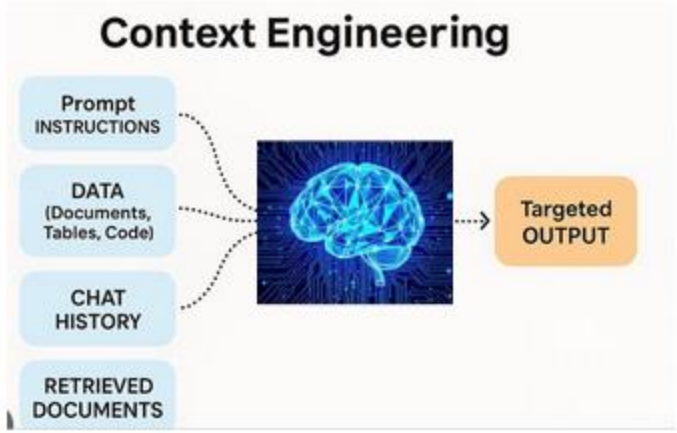


AI and software development

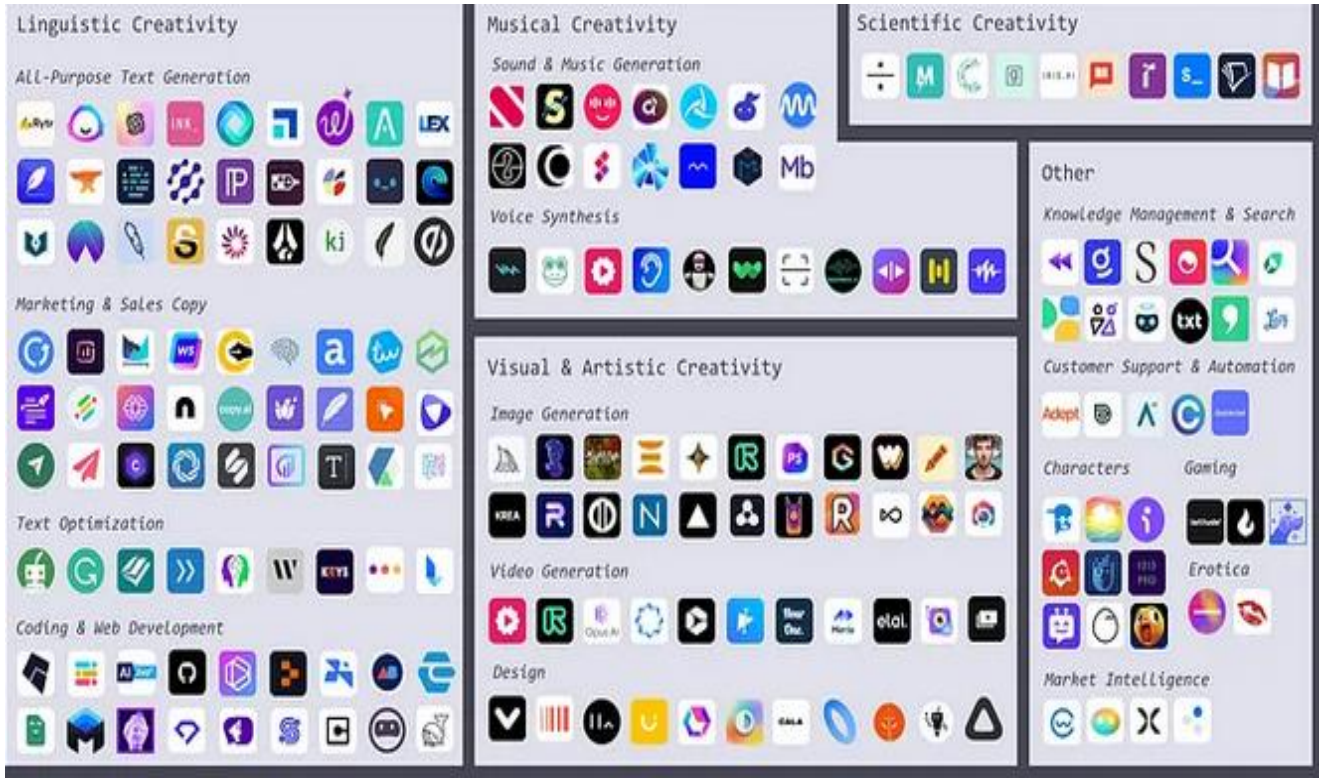
- Design diagrams
  - Generate diagrams from text
  - Generate diagrams from existing cod
  - Keep diagrams updated
- Writing code
  - Pair programming
  - Context-aware completion
  - Code generation
  - Filling boilerplate
  - Enforce best practices
- Code migration
  - Deprecated APIs
  - Third parties
  - Programming language version
- Optimization
  - Code level
  - Compiler/linker flags
  - Profiling and anomaly detection
  - Algorithmic refinement
- Project Setup
  - Environment setup
  - Build system setup
  - Dependencies
  - Initial draft
- Testing
  - Setup
  - Test cases generation
  - Test data generation
  - Clean test code
- Documentation
  - Generate Doc
  - Update Doc after code change
  - Document legacy code
- Refactoring
  - Naming
  - Detect smells
    - God classes
    - Long methods
    - Duplication
  - Enforce style/idioms
- Code search
  - From KW + RE to Semantic search
  - API Lookup Internal/External
  - Examples
  - Docs
  - Explanation
- Debugging
  - Stacktrace analysis
  - Logs and error messages
  - Generate root cause hypotheses
  - Pair debugging
- Code review
  - Enforce
    - code conventions
    - Best practices
    - language standards
  - Code suggestions from formatter, static/dynamic analyzers, and QA tools
  - Detect security vulnerabilities
- Experimentation
  - Fast feedback cycle
  - Quick prototypes
  - Try different
    - Libraries
    - Languages
    - Structure
    - Designs

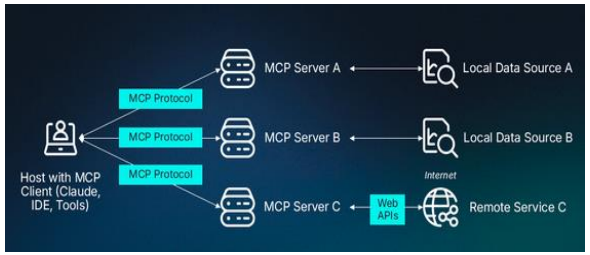
AI Tools and Agents Landscape

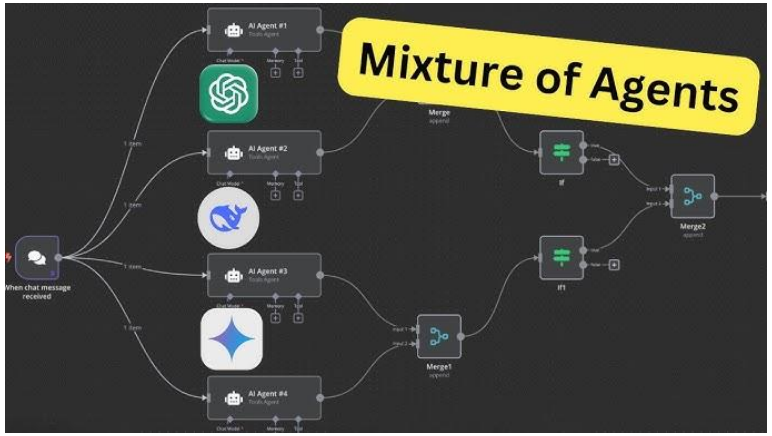
Prompt Engineering Guide





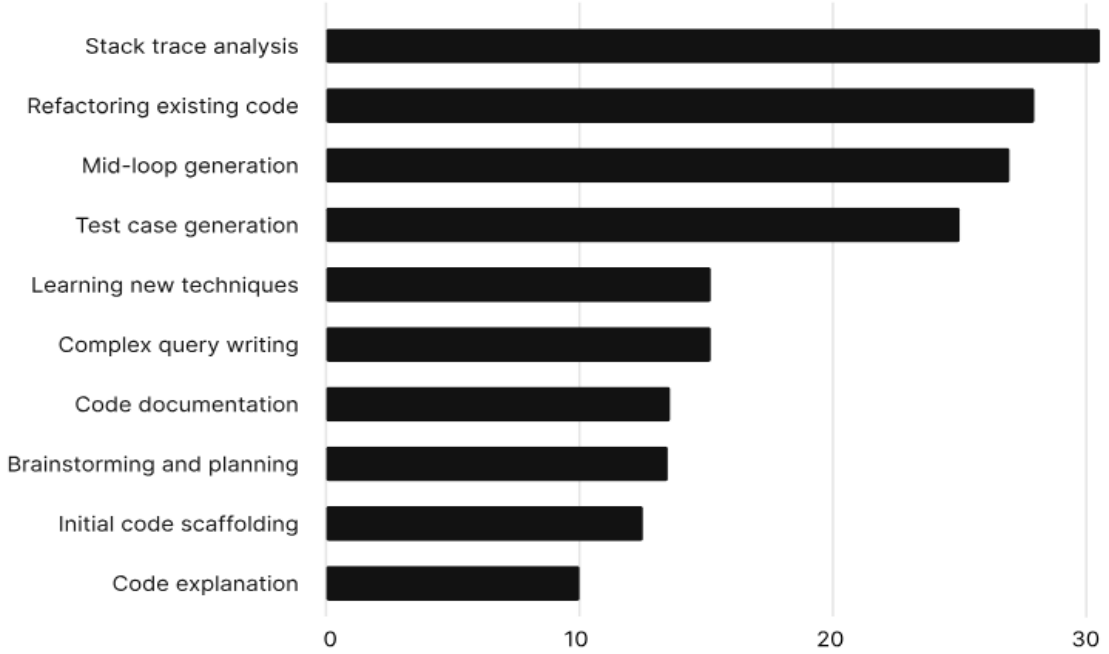








Getdx company survey report about the most time saving use cases for AI code assistants.

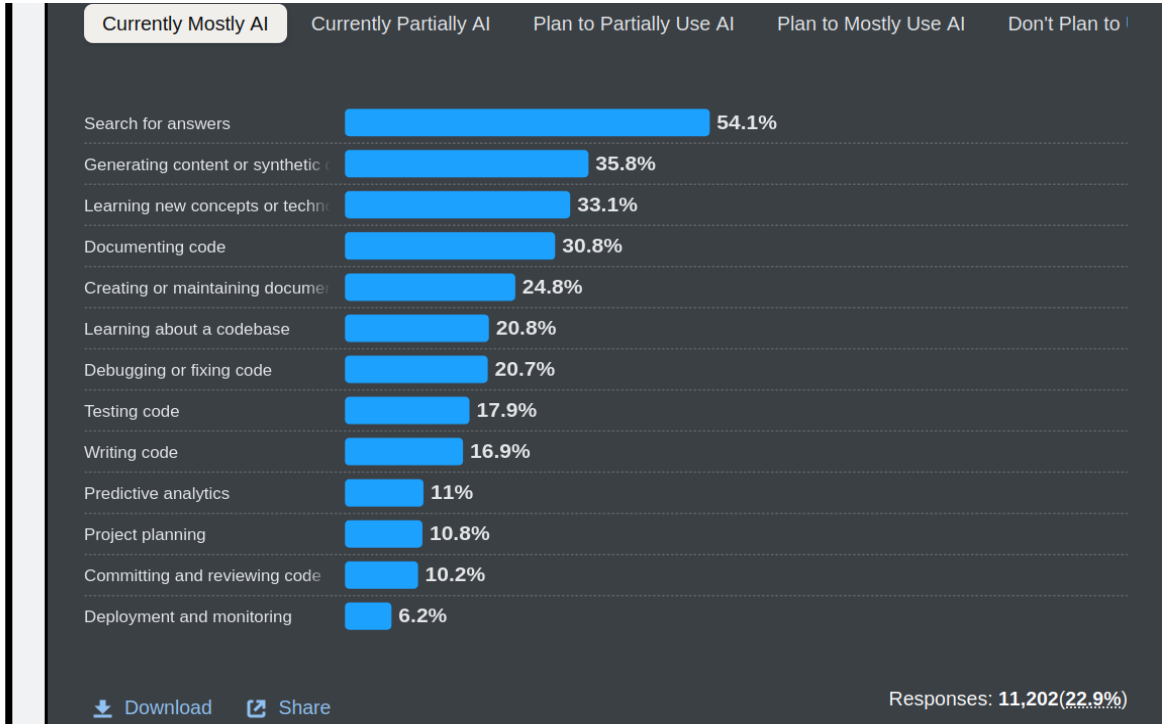


[Stackoverflow 2025 Developer Survey]

AI in the development workflow

Developers show the most resistance to using AI for high-responsibility, systemic tasks like Deployment and monitoring (76% don't plan to) and Project planning (69% don't plan to).

Which parts of your development workflow are you currently integrating into AI or using AI tools to accomplish or plan to use AI to accomplish over the next 3 - 5 years? Please select one for each scenario.



AI-Assistant best practices

- Start small and with defined scope
  - Decompose complex tasks into smaller calls
  - Use the right model for each subtask (specs, design, code)

- Use specific, explicit and clear prompts

- Use Meta-Prompt
  - Embed instructions withing the prompt
  - One/Few shot examples

- Add Multi-Context
  - Mention @dirs, @files, and @code-snippet
  - Documentation, images, and voice

- Use multiple models for the same task
  - Different perspectives and solutions
  - Make them evaluate each other answers



- Ask the model to self-critique/reflect
  - Catch flawed assumptions, inconsistencies, and hallucinations
  - Ask for refinement to get high-quality responses

- Check reasoning not just the output

- Always verify (You could have subtle bugs)

- Small change at a time

- Never share sensitive information

What does that mean for software engineering?

- Shift some focus from the how to what and why.
- Spend more time on design and architecture.
- Produce better code and better products.
- Solve more problems and realize more features.
- Speed up onboarding and familiarity with new codebases.
- Having tools that are smart and communicate well.
- Increase the sense of fulfillment and business impact.
- Tackle harder problems in all domains (Climate change, Energy, Health, Education).
- Increase accessibility to software development.
- Discover new algorithms or improvements to existing ones.
- New type of jobs (Prompt, Context, Agentic AI engineering)



Risks of using AI

- Hallucination & nonsense generation
  - Generate false information as fact
    False requirements to false test cases

- Data leakage
  - Sensitive information (passwords, API keys)
  - Proprietary algorithms
  - Intellectual property

- Compliance and IP concerns
  - Open-source licenses violation (GPL, MIT)
  - Use other's proprietary code

- cognitive offloading
  - Loss of deep understanding
  - Shallow learning loops
  - Critical thinking reduction



Don't blame me, I was trained on the good, the bad, and the ugly.
Don't forget, I am a program trained to predict the most likely next token . I don't care about the truth. Treat my answer as suggestions, not the final truth and you will get the best out of me.
I can make you smarter or more foolish. The choice is yours.